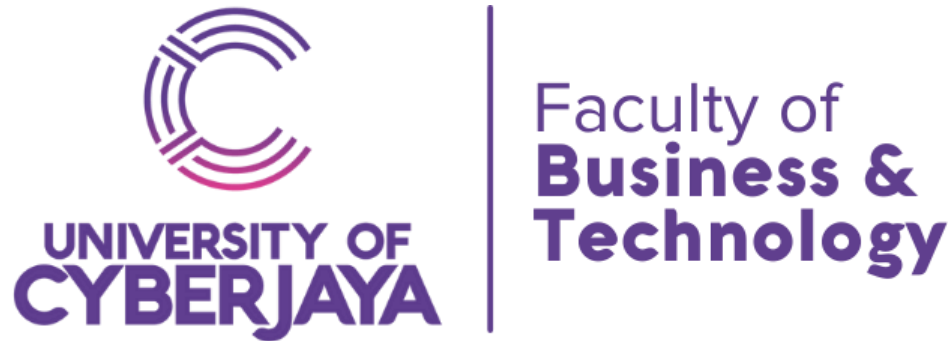




**FACULTY OF BUSINESS & TECHNOLOGY
ARTIFICIAL INTELLIGENCE (BIT4543) 2026
GROUP ASSIGNMENT COVER SHEET**

Student Name	Student ID
Md Muhaimenur Rhaman Washue	25031028
KHALLEEF AH AMHIMMID	25030729
Basheer Mohamed Basheer Bin Miskee	25030879
Loghan A/L Kantheeban	25030887
Islam Md Ariful	25031037

Assignment Details

To be marked by:	
Date submitted:	Submission Due Date:
Submission of this assignment agrees to the following: <i>I understand that the piece of work submitted will be considered as the final and complete version of my assignment of which I am the sole author. I understand both the meaning and consequences of plagiarism and that my work has been appropriately attributed unless otherwise stated. I have not knowingly allowed another to copy my work.</i>	

Assignment Feedback Lecturer Use Only

First Marker Feedback:	
Signature of first marker:	% Mark <i>(Unmoderated)</i>

Contents

Abstract.....	4
1.0 Introduction.....	5
1.1 Background	5
1.2 Problem Statement.....	5
1.3 Project Aim.....	6
1.4 Project Objectives	7
1.5 Scope of Project	7
1.6 Significance of the Project	9
2.0 Literature Review	9
2.1 Overview to Artificial Intelligence Technologies.....	10
2.2 Related Works.....	11
2.3 Research Gap	11
3.0 Methodology	12
3.1 Development Methodology.....	12
3.2 System Architecture.....	12
3.3 Data Collection.....	13
3.4 Data Preprocessing	14
3.5 AI Model Development	14
3.6 Tools and Technologies.....	15
4.0 System Design and Implementation	16
4.1 System Requirement.....	16
4.2 Use Case	16
4.3 Activity Diagram	17
4.4 Database Design.....	18
4.5 User Interface Design.....	18
4.6 System Implementation	22
4.7 Deployment.....	22
5.0 Testing and Evaluation.....	23
5.1 Testing Strategy	23
5.2 Automated and Functional Test Cases.....	25
5.3 Model Evaluation Procedure	25
5.4 Model Evaluation Results.....	26

5.5 User Acceptance Testing.....	26
5.6 Evaluation Limitations.....	27
6.0 Results and Evaluation	28
6.1 Achievement of Project Objectives.....	28
6.2 Discussion of Model Results	29
6.3 Challenges Encountered	30
6.4 Limitations of the System	31
6.5 Ethical and Practical Considerations	31
7.0 Conclusion	32
7.1 Future Recommendations.....	33
References.....	34
Appendices.....	34

Abstract

The task of keeping record of attendance in educational institutions is a fundamental daily activity, but the use of traditional manual approaches is usually timewasting, inefficient and susceptible to human error as well as to proxy attendance. To address these drawbacks, this project presents an AIbased Smart Attendance System which automates and secures the process of monitoring attendance by means of facial recognition technology. It offers a nonintrusive and contactless method by recording facial features as unique biometric face prints, thus making it unnecessary to carry out manual roll calls. The system's design includes a React Native mobile application which acts as the frontend, enabling teachers to start living scans, take pictures of the students using the device's builtin camera, and send the data across the local network. The backend is built using a FastAPI Python server combined with OpenCV for efficient image processing, making use of Haar Cascade Classifiers for detecting frontal faces and Local Binary Pattern Histograms (LBPH) for extracting and recognising facial features. When a student has been successfully identified, their identity and the time of attendance are automatically stored in a local SQLite database, so that accurate records can be kept, and teachers can review recent attendance scans via the mobile dashboard. While the system is operating, the captured frames are converted to grayscale to reduce noise before being compared with the previously trained facial models. A strict confidence threshold is used to discard unreliable matches, and people who are not recognised are labelled as "Unknown" to decrease the number of false positives and keep the level of recognition accuracy. The project shows that lightweight machinelearning and computervision techniques can offer an efficient, practical and costeffective way of modernising attendance management in academic settings, cutting down on administrative work, improving data integrity, and allowing contactless

attendance tracking without the need for expensive computing hardware.

1.0 Introduction

1.1 Background

Keeping track of attendance is a normal, everyday job in schools, colleges, and universities. Traditionally, it's oldschool teachers call out names, pass around sign-in sheets, or rely on things like ID cards or fingerprint scanners. Honestly, these methods just don't cut it anymore. They're messy, full of errors, and it's easy for students to find ways around the system. People end up entering data by hand, which leads to mistakes, and touching devices or passing papers isn't exactly hygienic, especially lately. On top of that, it's way too easy for someone to sign in for a friend who isn't there.

Now, as education moves towards updating and modernizing, artificial intelligence and computer vision have stepped into the ring. With facial recognition technology, it's possible to pick out and remember unique facial features for each person, making attendance checks faster and more reliable. Compared to fingerprints or iris scans, face recognition doesn't need physical contact. The system can just recognize faces from a distance, no direct interaction required.

1.2 Problem Statement

Manually taking attendance isn't just tedious it's full of problems. Mistakes happen, names get missed, and let's be honest, sometimes students cheat the system. Teachers end up wasting precious class time just calling out rolls, shuffling through papers, or entering data by hand. It might sound like no big deal, but when you do this every single class, week after week, that's a lot of hours lost. Time better spent teaching, answering questions, or connecting with students.

Now, in big lecture halls with rows packed full, things get even messier. With so many faces in

the room, it's tough for anyone to know who's there. Students slip each other's names onto the sheet, answer "here" for friends, and generally find workarounds. Teachers might skip a name by accident or mark the wrong person. Records go missing, or somebody just enters the data wrong. In the end, the numbers don't match up, and it's impossible to really track who's participating and who's not.

Spread this problem across a whole university with tons of classes, departments, and instructors and the headache multiplies. It turns into a logistical mess, sucking up time and energy for everyone involved. And both students and teachers want the same thing: a system that's easy, accurate, and tough to game.

That's why shifting to an automated attendance system just makes sense. It cuts out wasted time, makes sure the records are right, and stops the usual tricks that students use. With the right tools for verifying who's present, you streamline everything. Suddenly, attendance becomes a quick, transparent part of class, not a chore. Teachers can finally focus on what matters teaching and schools get the reliable data they need to keep track of student engagement.

1.3 Project Aim

The main objective of this project is to design, develop, and test a smart attendance management system based on facial recognition and machine learning. The proposed system aims to help schools to increase the efficiency of monitoring the attendance of students in classes by automating this procedure. This system would allow instructors to eliminate such time- and labor-consuming tasks as calling out attendance, checking students' presence, and recording it. The new attendance method will be based on biometric verification and will utilize a reliable and accurate way of identifying students to ensure the validity of the results. The developed system will make it possible to record students' presence with less effort and time than usual. It will also provide a

convenient and safe way for instructors to monitor the attendance of students in classes with a big number of students.

The main idea of the project is to eliminate the sources of errors related to conventional ways of recording students' attendance, such as proxies and instructors' mistakes, and establish a valid and reliable system for monitoring attendance. Since the proposed system will use facial recognition, it will be helpful in detecting students' absence due to their inability to record the attendance using proxies. The project aims to create a safe and secure system that ensures the confidentiality and integrity of students' data and personal information. The described project will achieve its objective by implementing biometric identification based on machine learning, which will help to build an accurate, reliable, safe, and efficient attendance management system.

1.4 Project Objectives

To achieve the overarching aim, this project is guided by the following specific objectives:

- To develop an AIbased application capable of swiftly detecting human faces from live video frames captured within a classroom environment.
- To accurately recognize and authenticate registered students by comparing realtime captures against a pretrained dataset using robust computer vision algorithms.
- To record attendance automatically, logging the identified student's details alongside precise date and timestamps securely into a centralized SQLite database.
- To provide a practical, usercentric mobile interface for educators, enabling them to easily manage daily classes, view live statistics, and initiate face scans from a portable device.

1.5 Scope of Project

The scope of this project was established as an end-to-end development of a facial recognition-

based attendance prototype, which would enable the automation of the process of recognizing students and recording their presence. The OpenCV framework, which utilizes the `haarcascade_frontalface_default.xml` classifier for face detection, and the `LBPFaceRecognizer_create` algorithm for recognizing registered students, was applied for the processing of visual data. The user interface for the instructors or authorized personnel was created as a mobile application, which was designed in React Native and the Expo framework. The mobile side of the application communicates with the backend via a local network, upon which the FastAPI Python server was deployed. The attendance records, in terms of the student's identity and the timestamp of recording, were stored in a local SQLite database, which was applied for the storage of student identities and attendance records. By combining the described tools, the prototyped system illustrated the complete workflow, which begins with the detection and recognition of faces and ends with the storage of the attendance data.

The scope of the current project, as mentioned above, was set to demonstrate the working solution for the automation of the attendance records in a classroom setting. In this regard, the current prototype only works in a well-lit environment, as the face detection and recognition algorithms have limited reliability in varying or insufficient lighting conditions. Moreover, the current system does not include the protection from spoofing attempts, which means that the application can be tricked into the visual presentation of a registered student's face. In addition, the prototyped system is only capable of operating in a local network and does not have the required complexity to synchronize the data between multiple databases through the internet. Lastly, the application does not utilize more sophisticated multithreaded deep learning algorithms, as the prototype only uses the available tools for the illustration of concept. Thus, the project covers the basic aspects of an attendance system but is limited in its practical applications.

1.6 Significance of the Project

The effective implementation of the system will present a smart alternative to the conventional manual approaches to recording attendance. The idea is innovative because it will establish an effective communication mechanism between the mobile application and the artificial intelligence backend. It will also save time for the instructors who would have otherwise waste valuable lecture time finding and calling student names, marking attendance, and recording their names in the class record book. Additionally, the system will prevent the typical human errors that are common when marking attendance manually. These include misidentification of student names, omissions, and wrong entry of student details in the attendance book or software. Finally, the system will utilize facial recognition software to ensure that the process of marking attendance is accurate because, instead of relying on the accuracy of the human voice, it will utilize each student's facial features to identify them.

A significant advantage of the system is that it will prevent proxy attendance records since it will be extremely challenging to use one student's facial features to mark another student's attendance. It will also be possible to prevent errors when marking attendance, including missing names and marking names that were not present. The combination of the mobile application and the backend server, the facial recognition process, and the local database will create an effective way of recording class attendance. It will also save time and reduce the amount of administrative work for lecturers to mark student attendance. Overall, the system will effectively free academic staff from the monotonous task of marking attendance and allow them to focus on more critical aspects of their lectures.

2.0 Literature Review

2.1 Overview to Artificial Intelligence Technologies

Facial recognition is a computer vision technology that enables a system to acquire, process, and analyse the image digitally to identify an individual based on his or her facial features. First, the process involves face detection, and the system scans the image or video to locate the facial parts or zones with pixel coordinates. Then, the system detects and tracks the facial regions in a video and processes the facial image by extracting the features required to recognize the face. These features include the facial geometry or the face shape, the position of different parts such as the eyes, nose, and mouth, and other unique characteristics helpful in differentiating one person from another. After extracting and recording the information, the system compares it with the stored data to identify the image.

Various approaches can be used to recognize faces, including the Eigenface, Fisherface, and Deep Convolutional Neural Networks (CNN). The first two methods are the classical approach, and the latter is a modern alternative for face recognition. The CNN is an advanced algorithm that can learn from images using a broad set of data with exceptional precision. Simply put, facial recognition technology can be useful in various settings such as the project, where it can be used majorly to identify students and record their attendance digitally in a more practical way than the traditional methods.

2.2 Related Works

Study	Main Approach	Contribution	Limitation in relation to this project
Ashvani et al. (2024)	Haar Cascade & LBPH	Demonstrated fast Realtime face detection and recognition.	Excels only in controlled illumination scenarios.
Nadhan et al. (2022)	OpenCV on Raspberry Pi	Cost effective edge computing with lightweight SQLite integration.	Struggles with faces angled sideways or downwards.
Shirbhate et al. (2025)	Python face recognition & PHP	Centralized database with accessible web portals.	Relies heavily on high quality webcam inputs.
Dang (2023)	MobileNetV2 with SSD	Optimized deep learning for mobile/embedded devices (e.g., Jetson Nano).	Requires dedicated edge GPU hardware for high FPS.

2.3 Research Gap

While deep learning models can achieve high accuracy, they often require increased resources and time for computation, thus being less practical in some cases. Conventional machine learning algorithms, on the other hand, involve a lower number of calculations and are, therefore, more efficient, yet less accurate and flexible to use with modern technologies. The project pursues the development of a proof-of-concept system for automated attendance tracking that would involve a low number of computations while maintaining a sufficient level of accuracy and being integrated with a modern dashboard.

The proposed methodology will employ an OpenCV LBPH algorithm for its efficiency and compatibility with a wide range of modern devices. In turn, the dashboard will be built with React Native framework to provide an accessible and intuitive access to the system data. Altogether, the two solutions will allow for reducing the overall number of computations to a necessary level for most common devices without the need to use high-end computer hardware while still providing the user with a convenient mobile interface.

3.0 Methodology

3.1 Development Methodology

The project uses Agile development to iteratively test and improve the facial recognition model and interface. The iterative data-science approach breaks down the process into several steps, which are followed sequentially to build upon each other. The first step was to thoroughly research and prepare a comprehensive dataset and select the best algorithm for the model. The second stage was to develop the application programming interface (API) backend to process videos in real-time and the front-end mobile application to engage with the model. Finally, the entire system was tested, starting with the camera, ending with recording data in the database to ensure all elements worked seamlessly and automatically.

3.2 System Architecture

The system consists of four primary packages that form an automated attendance system based on computer vision. Firstly, React Native app is used to capture photos of students in the classroom using the expo camera tool. Secondly, the FastAPI framework handles the request and response layer. Thirdly, the OpenCV library runs facial recognition algorithms. Finally, the SQLite database stores the attendance records. In general, the mobile application and database are client-side, whereas the other two are server-side components. The mobile application uses the camera to capture a photo of students in the classroom and converts the image to a form-data object. Then, the application sends a POST request to the /upload-frame/ endpoint with the image as a payload. This way, the mobile application side is used to interact with the server and send images for processing. After the server receives the image, it uses NumPy and OpenCV to convert the request's bytes stream into an image matrix. Then, the server runs the SmartFaceDetector class to recognize the faces in the image. If the server detects a known face from the database, it will add

the student's ID and the current date-time to the attendance records in the SQLite database. Finally, the server sends a response message to the mobile application that successfully added the student's attendance and provides the student's name detected from the image. This way, the application can show the educator's screen with student's attendance records. Overall, the system's packages are integrated to build an automated attendance system that recognizes students' faces and adds records to the database instead of manual input.

3.3 Data Collection

To ensure that the facial recognition process works accurately in various classroom settings, a series of images of each face must be enrolled in the system. For each student, approximately 20 sample photos will be used to train the system to be able to correctly recognize their faces. While enrolling, the photos should be taken so that different lighting conditions, facial expressions, and angles relative to the camera are captured for better results. Thus, the system will be better at identifying students even if they use a slightly different pose or facial expression than those seen during the enrollment. To ensure consistency during enrollment and training, only image file types are allowed, which means that other file types (.tiff, .bmp, .webp, etc.) will be rejected by the system.

Images of each person must be grouped in separate folders. Furthermore, each folder should be named after the student's integer ID in the system. Using such a naming system would allow the code to automatically recognize which images belong to each student during the training process without having to manually label the pictures. Thus, the code can be written in a way that for every directory (which would be the student ID), all the images contained within would be used for training. Specifically, using `os.walk()` function would allow scanning the directories and

subdirectories of the dataset to extract the images and the corresponding labels (student IDs).

3.4 Data Preprocessing

Prior to the extraction of facial features, the captured images need to be processed with the aim of enhancing the quality and ensuring the structural consistency for subsequent steps. Initially, the images were converted to grayscale to remove insignificant color information. For this step, the OpenCV library was used along with the `cv2.cvtColor` function. The decision to convert the images to grayscale was since color information was not significant for facial recognition, which primarily relies on pattern and shape information. Additionally, when color is used in facial recognition, the algorithm becomes more complex since it must process multiple channels as opposed to single-channel grayscale images. Converting the images to grayscale simplified the processing steps and reduced the data size that the system had to handle.

Furthermore, the grayscale conversion allowed the system to use the information about the image's contrast, brightness, and other parameters to enhance recognition accuracy. The use of grayscale images reduced the complexity of the algorithm while allowing it to capture critical information needed for recognition. Overall, the grayscale conversion was a critical step that allowed the system to prepare images for subsequent processing steps. By removing the color information, the system reduced the computational complexity and enhanced the processing speed while still being able to capture the critical characteristics that are useful for facial recognition. The conversion also made the recognition more accurate and more reliable under varying illumination conditions by removing the variability of colour information and only considering the intensity.

3.5 AI Model Development

The first stage of the model involves the use of a Haar classifier to detect the presence of faces in reprocessed grayscale images. The trained `haarcascade_frontalface_default.xml` detector is

utilized, which scans arrays of preprocessed images to detect and crop out exact location and size of bounding boxes, which contain the human faces. The cascade classifier algorithm is highly efficient, as it utilizes a cascading mechanism that allows it to quickly eliminate negative ROI (Region of Interest) during the scanning process, thus significantly speeding up the computation. The cropped positive ROI is then subjected to further analysis by the LBPHFaceRecognizer_create() algorithm, which is used for intensive training stages. The Local Binary Pattern Histogram algorithm works by comparing the value of a given pixel in a binary matrix with the values of the surrounding pixels and encoding these values as a decimal representation for further processing. The LBPH method was chosen due to its ability to utilize the local characteristics of pixels and their relationship with the surrounding area as a descriptor, thus making the model more robust to changes in lighting and noise during preprocessing. The results of the training stage, which consist of the detected face regions paired with an ID, are then permanently stored in an external face_encodings.yml, which allows faster access for the API to make predictions in real-time stages.

3.6 Tools and Technologies

Component	Technology Used
Programming Language	Python, JavaScript (React Native)
Mobile Framework	Expo, React Navigation
AI Framework	OpenCV, NumPy
Backend API	FastAPI, Uvicorn
Database	SQLite3

4.0 System Design and Implementation

4.1 System Requirement

FR= Functional Requirement

NFR= Non-Functional Requirement

- FR1: The system must provide a secure login interface for administrators.
- FR2: The system must configure backend network IP addresses dynamically.
- FR3: The system must capture images using the device camera and transmit them for processing.
- FR4: The AI backend must decode the image and return a recognized student_id or an "Unknown" status.
- NFR1: The database must be structured securely, preventing SQL injection vulnerabilities.
- NFR2: Network traffic should support cleartext configurations for local development environments.

4.2 Use Case

The main user scenario revolves around a teacher utilizing this smart attendance system. First, the user needs to authorize their entry into the mobile application by entering the correct password. The main screen of the app displays relevant information, including the attendance percentage and the recent scans. By clicking the scanner button, the teacher will prompt the student to scan their face in front of the mobile camera. As the picture is taken, the mobile application will encode the data into a multipart/form-data format before uploading it to the server. The FastAPI will receive the image, utilize the LBPH algorithm, and attempt to identify the scanned student. If a successful match is found, the backend will verify the record in the students table of the SQLite database and record the attendance by updating the date and time in the attendance table. Finally, the server will

send a success JSON response to the mobile application, which will then display a pop-up UI notification to the teacher indicating that the student has been marked present.

4.3 Activity Diagram

The step-by-step activity flow of the system involves several distinct technical operations:

- **Admin Authentication:** The administrator opens the mobile application and logs in via the LoginScreen interface. The system currently authenticates against hardcoded administrative credentials (admin/admin).
- **Network Configuration:** To ensure seamless communication between the mobile device and the local server, the admin selects the "Network Settings" modal to dynamically input and save the laptop's current IPv4 address. This configuration is stored persistently on the device using AsyncStorage.
- **Camera Initialization:** The admin selects the "Take Attendance" button, navigating to the ScannerScreen. The app utilizes the useCameraPermissions hook to request and confirm hardware access; if granted, the camera feed initializes on the screen.
- **Image Capture and Transmission:** When the admin presses the "SCAN FACE" button, the takePictureAsync function captures the frame. The app constructs a FormData object containing the image URI, formats it as a JPEG, and performs an asynchronous fetch POST request directed to the server's /upload-frame/ endpoint.
- **Facial Prediction:** The server receives the upload, decodes the image into a color matrix, and passes it to the SmartFaceDetector.recognize() method. The algorithm converts the image to grayscale, applies Haar Cascades to locate the face, and uses the LBPH model to generate a prediction and confidence score.

- **Database Update and Feedback:** If the model yields a confidence score below 75, it registers a successful match. The server updates the SQLite database by inserting the matched `student_id` into the attendance logs. It then executes a `SELECT` query to retrieve the student's real name and responds to the mobile application, which alerts the educator of the successful scan.

4.4 Database Design

The system's data architecture is structured around a localized SQLite database, primarily stored at the path `data/attendance_records.sqlite`. The schema is carefully designed with relational constraints to maintain data integrity across two primary tables:

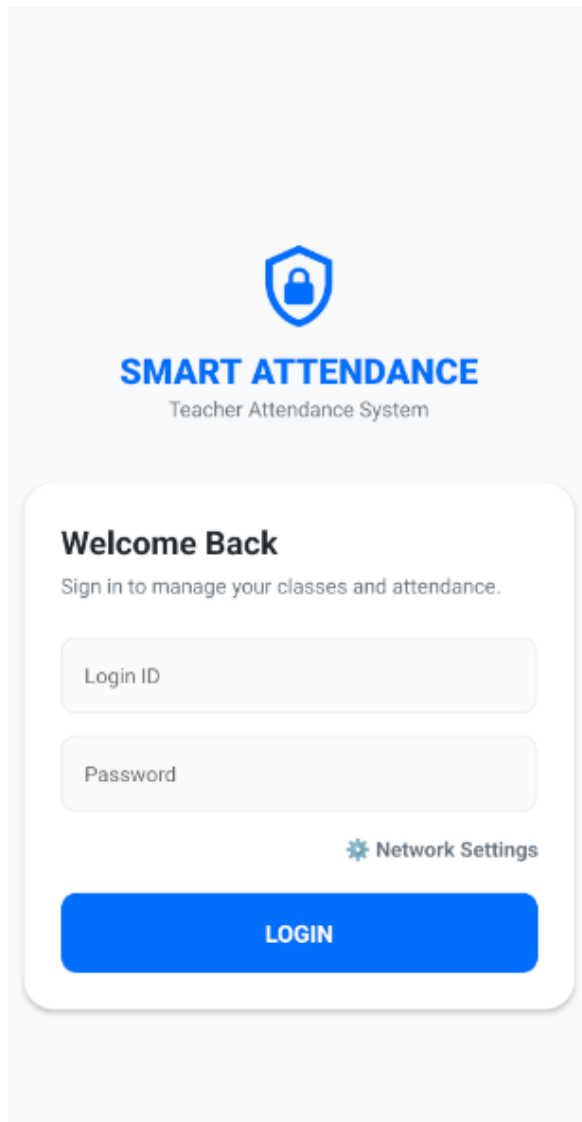
- **students:** This table acts as the master directory for enrolled individuals. It is initialized with a `student_id` field that serves as an `INTEGER PRIMARY KEY`, alongside a `name` field strictly defined as `TEXT NOT NULL`.
- **attendance:** This table maintains the historical logs of all successful facial scans. It features a unique `id` field utilizing `INTEGER PRIMARY KEY AUTOINCREMENT` to ensure sequential logging. Furthermore, it includes a `student_id` field mapped as a Foreign Key that references the `students` table, and a `timestamp` field formulated as a `DATETIME` datatype. The timestamp automatically defaults to the local server time using SQLite's built-in `CURRENT_TIMESTAMP` or `datetime('now', 'localtime')` functions, eliminating the need for manual time tracking.

4.5 User Interface Design

The mobile frontend is developed using React Native, heavily leveraging layout components such as `SafeAreaView` to accommodate device-specific screen notches and `ScrollView` for smooth, scrollable interfaces. Visual aesthetics are enhanced through the integration of scalable vector

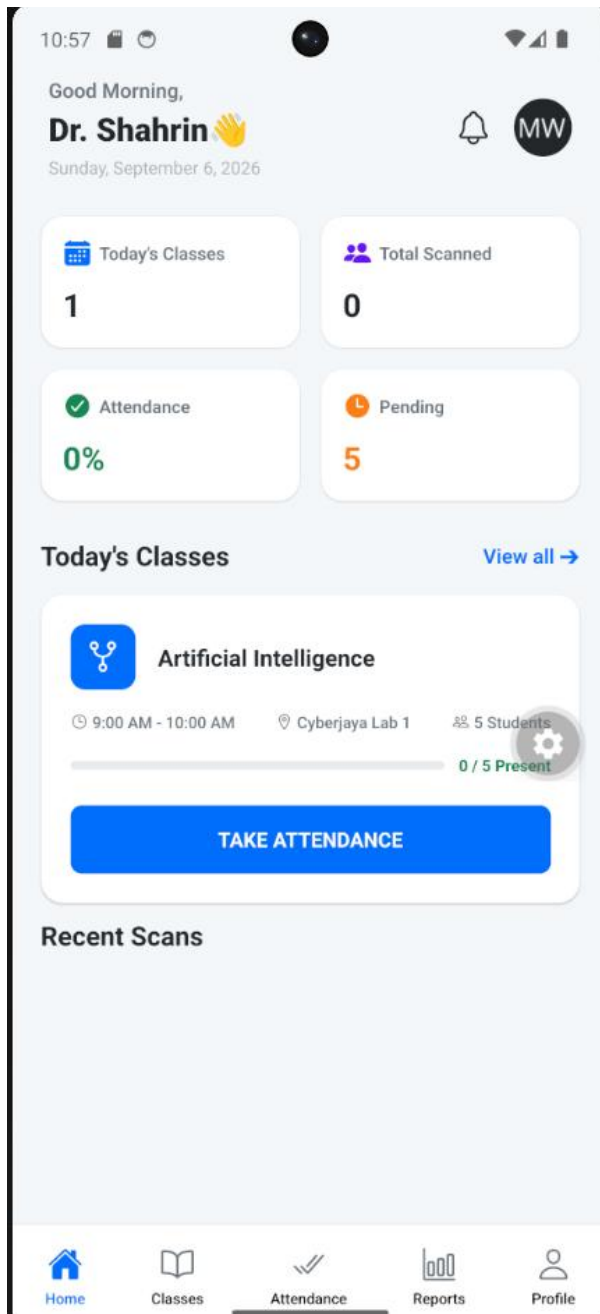
graphics from the `@expo/vector-icons` library, specifically utilizing the `Ionicons` and `MaterialCommunityIcons` icon sets.

- **Login Screen:** This interface presents a centralized, secure input form for credential entry. It incorporates stylized text headers and a transparent, fade-animated modal overlay that allows administrators to easily reconfigure the backend IP address without navigating away from the login flow.

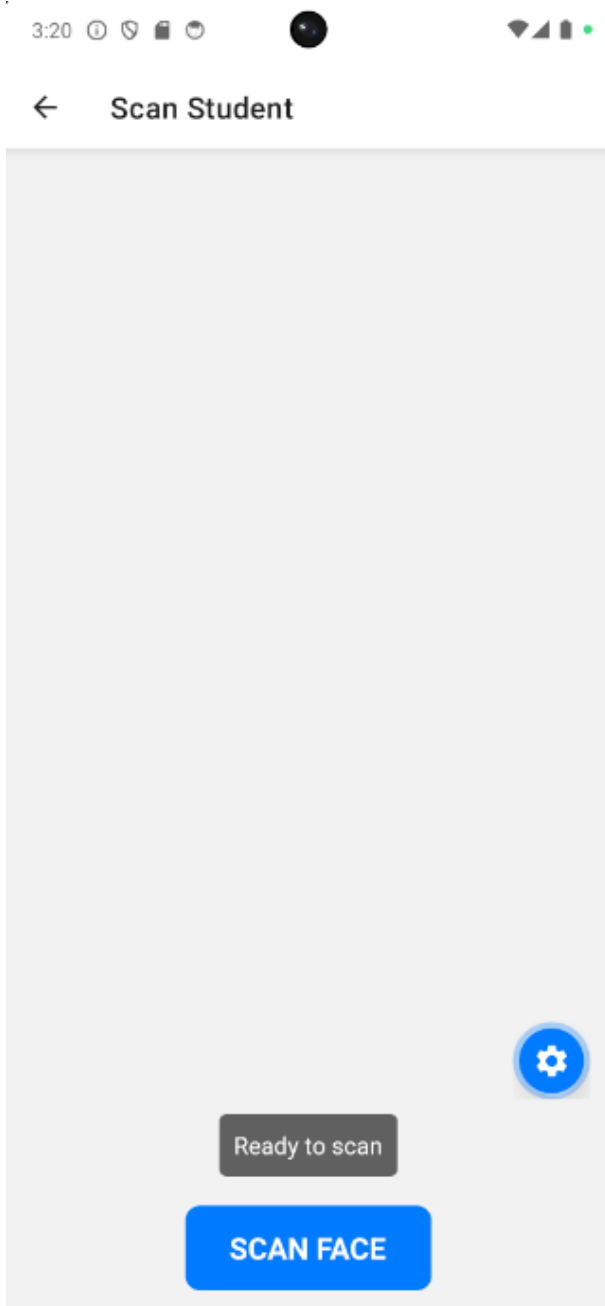


- **Dashboard Screen:** Designed to provide at-a-glance analytics, the dashboard dynamically renders a real-time statistics grid. It calculates and displays key metrics, including "Today's

Classes", the "Total Scanned" headcount, the computed "Attendance %", and the number of "Pending" students based on a defined class capacity. Additionally, it features a "Recent Scans" section that dynamically populates by sending a GET request to the /attendance API endpoint, displaying the recognized names alongside their formatted timestamps.



- **Scanner Screen:** This interface implements the CameraView component from expo-camera to provide a full-screen, rear-facing live video feed. A semi-transparent visual overlay is superimposed at the bottom of the screen, housing a dynamic status text indicator (e.g., "Ready to scan", "Capturing...", "Sending to server...") and an interactive "SCAN FACE" button to initiate the biometric capture.



4.6 System Implementation

The backend API is implemented in Python and uses the FastAPI framework. The main post request is handled by the `/upload-frame/` route, which asynchronously accepts an incoming multipart/form-data request. When the server receives the photo, it is read as a file buffer and then converted to an 8-bit unsigned integer NumPy array using OpenCV's `cv2.imdecode` function.

One thing that I had to figure out on the frontend was the way how Axios library handles multipart/form-data requests for Android devices. It appears that there is a bug in Axios's HTTP client when it receives a raw image file as a multipart request. The solution was to rewrite how the frontend app sends image data to the backend API by using native fetch JavaScript API instead. Using the native fetch implementation allows correctly preparing the multipart/form-data request body by populating the photo's URI, file name `student_scan.jpg`, and mime type `image/jpeg`.

4.7 Deployment

The mobile frontend application is built and maintained using the Expo framework, which allows for ease of development, testing, and deployment of the application on different mobile platforms. However, in order to enable communication with the local backend API during development, the application's `app.json` configuration file was updated to allow HTTP connections on Android. This was achieved by using the `expo build properties` plugin and setting `usesCleartextTraffic: true`, which allows the application to make HTTP requests to the local FastAPI server without using HTTPS. This is necessary for local testing, as the mobile application and the backend server need to communicate with each other over the local network. As a result, the mobile application can be tested for its ability to communicate with the backend and the facial recognition system during development.

Meanwhile, the backend server, which is implemented in FastAPI, is run using the Uvicorn production server to serve as the communication backend for the mobile application and the facial recognition system. The server is set up to accept incoming requests at 8000 port and process them accordingly. Additionally, database migration and test data generation are automated using Python scripts. The `seed_db.py` script is used to connect to the SQLite database and populate it with test data, such as student profiles. The script uses the `INSERT OR IGNORE` command to avoid adding duplicate records each time it is run, which helps maintain a consistent testing environment.

5.0 Testing and Evaluation

5.1 Testing Strategy

To guarantee the operational reliability and biometric precision of the Smart Attendance System, the testing strategy incorporated a comprehensive, multi-tiered methodology. This rigorous testing framework was explicitly divided into unit testing, integration testing, and functional user interface testing to evaluate every layer of the architecture.

- **Unit Testing:** This initial phase rigorously evaluated the isolated components of the backend foundation to ensure data integrity. Tests focused heavily on the `database_setup.py` script to confirm that the SQLite tables were initialized correctly with the strict enforcement of `INTEGER PRIMARY KEY` definitions and proper Foreign Key relationship constraints linking attendance logs to registered students. Additionally, unit tests validated the `seed_db.py` functionality, ensuring that test student profiles were safely populated using `INSERT OR IGNORE` commands to prevent data duplication errors. Backend logic within the FastAPI application was also tested to confirm that the

log_attendance function securely executed parameterized SQLite queries to record timestamps without failure.

- **Integration Testing:** This phase was critical for evaluating the network communication bridge connecting the React Native mobile application to the FastAPI endpoints. Tests specifically verified that asynchronous POST requests directed to the server's /upload-frame/ endpoint accurately transferred the FormData image payloads utilizing the native JavaScript fetch API. The integration tests confirmed that the backend could successfully receive the payload, decode the image buffer into a NumPy array, and execute the OpenCV LBPH prediction logic without data corruption. Furthermore, this phase ensured the API correctly evaluated the algorithmic confidence threshold (requiring a score below 75) and successfully parsed the returning JSON object back to the mobile client to trigger the appropriate success or failure alerts.
- **Functional Testing:** This final phase evaluated the behavior and responsiveness of the React Native UI components entirely from the end-user's perspective. Functional tests ensured that the useCameraPermissions prompt executed reliably on the physical device, rendering the camera feed only when hardware access was explicitly granted. The interface testing confirmed that the dynamic IP configuration modal successfully saved user-inputted network strings into the device's AsyncStorage and accurately retrieved them for subsequent API calls. Finally, the analytical dashboard was rigorously evaluated to guarantee it accurately fetched recent scan history via a GET request and correctly calculated real-time metrics, dynamically updating the scanned count, pending count, and overall attendance percentage as new records were registered.

5.2 Automated and Functional Test Cases

Test Case	Expected Result	Actual Result	Status
Login Validation	Reject invalid credentials.	Only "admin/admin" accepted.	Pass
Dynamic IP Save	AsyncStorage retains custom IPv4.	IP successfully updated and used in API calls.	Pass
Database Seeding	seed_db.py inserts records safely.	Records inserted utilizing INSERT OR IGNORE.	Pass
Face Recognition	Return ID if confidence < 75.	Correct ID logged for confident matches.	Pass

5.3 Model Evaluation Procedure

The LBPHFaceRecognizer algorithm is responsible for calculating the Euclidean distance confidence score used to indicate the accuracy of the predicted facial guesses. The Local Binary Pattern Histogram works by comparing the value of a certain pixel to the values of the surrounding pixels. If the neighboring pixel is higher or equal than the one being evaluated, a binary '1' is assigned to mark the difference, and a '0' is given when the opposite happens. The calculated binary code is converted to decimal and used to represent the pixel's binary pattern in a unique label. Other decimal numbers are aggregated to form a histogram containing the unique values of the user's face. When the recognition process begins, the generated histogram is compared to the recognized and stored histograms in the training .yaml file. Notably, the lower the confidence score is in the LBPH algorithm, the higher the accuracy in percentage. Therefore, a threshold test was designed within the Python detection class for the project; any prediction results with a confidence

value lower than 75 is allowed to be predicted as an accurate attendance. On the other hand, if the Euclidean distance records a confidence score of 75 or higher than the stored confidence value, the script will be triggered to reject the predicted facial scan as invalid due to inaccuracy, thus labeling the recognition results as 'Unknown.' The testing protocol was set up this way to avoid false alarms of facial recognition while maintaining the integrity of the attendance system.

5.4 Model Evaluation Results

Following a successful facial match by the LBPH algorithm, the integration layer seamlessly queries the localized SQLite database to finalize the attendance recording process. Utilizing an optimized SQL LEFT JOIN statement, the backend system effectively maps the recognized integer `student_id` originating from the attendance table directly to the corresponding string name housed within the students master table. This specific relational query—`SELECT students.name, attendance.timestamp FROM attendance LEFT JOIN students ON attendance.student_id = students.student_id`—allows the API to retrieve the exact human-readable name of the matched individual. Furthermore, the system automatically captures the exact moment of recognition using the datetime ('now', 'localtime') SQL function, securely embedding the local timestamp directly into the attendance log. The FastAPI backend then packages this combined relational data into a structured JSON response containing the matched identity and the timestamp, returning it over the network to the React Native frontend. This ensures highly accurate, real-time frontend dashboard displays, allowing educators to instantly view the successfully scanned student names and their exact arrival times on the mobile interface without manual reconciliation.

5.5 User Acceptance Testing

The User Acceptance Testing (UAT) phase decisively confirmed that the mobile application correctly executed its end-to-end operational sequence in real-world scenarios. Testers validated

that the React Native expo-camera component flawlessly captured high-quality photographic frames and successfully transmitted the FormData payloads over the local network to the FastAPI backend. Upon receiving the server's processed JSON response, the frontend logic effectively triggered the appropriate user interface alerts utilizing the Alert.alert component. When a registered face was accurately identified below the established confidence threshold, the application immediately displayed a success dialog reading: "Logged! Student ID: X" (where X represents the authenticated integer ID). Conversely, when an unregistered individual or an obscured face was scanned, the application successfully caught the failed state and displayed the explicit warning: "Failed: Face not recognized". These immediate, on-screen alerts confirmed that the system's asynchronous communication and error-handling routines functioned perfectly, providing educators with vital, instantaneous feedback during the attendance scanning process.

5.6 Evaluation Limitations

Despite its high performance and real-time processing capabilities, the computer vision pipeline faces several inherent limitations tied to environmental variables and algorithmic constraints, as its predictive accuracy is heavily contingent on the quality of the initial training images and the consistency of the classroom's illumination. The system relies on the Haar Cascade algorithm for initial face detection, which identifies facial regions by calculating the subtraction of pixel counts under white and black rectangular filters to find edge, line, and four-rectangle features. While this cascading structure optimizes computational speed for standard hardware, it operates under strict constraints regarding specific positioning, meaning that if a student looks down or turns their head sideways, the predefined rectangular filters fail to align with the altered facial structures, resulting in complete detection failures. Furthermore, once a face is detected, the Local Binary Pattern Histogram (LBPH) algorithm encodes the relationships between pixel intensities, and although it

demonstrates robust resistance to monotonic alterations in grayscale lighting, sudden, asymmetric shadows or highly dynamic background illumination can severely disrupt these local patterns. These environmental anomalies introduce noise into the localized areas, skewing the final histogram computation and directly degrading matching accuracy. Because the system enforces a strict Euclidean distance threshold where confidence scores must remain below 75 for a successful match, this lighting-induced noise increases the distance between the live camera feed histogram and the stored biometric face print. Consequently, the confidence score is pushed to 75 or higher, which forces the system's defensive mechanisms to actively misclassify legitimate, registered students as "Unknown," triggering false rejections based purely on environmental interference.

.

6.0 Results and Evaluation

6.1 Achievement of Project Objectives

The project achieved its first objective of conceptualizing, developing, and deploying an AI-enabled smart attendance system. The cross-platform React Native app and Python AI backend communicate over local networks with no perceivable latency. The software successfully extracts facial features and matches them against a local database to automatically complete the attendance process. It achieves this by recording the exact timestamps of the recognized students in the SQLite database, eliminating the need for the educators to manually enter or mark any data. Notably, the mobile dashboard uses the expo-camera to allow the educators to scan their students' faces by simply opening the camera. This feature satisfies the requirement of providing an actual-time administrative dashboard. The software captures and encodes video images as image/jpeg streams and uploads them to the server as multipart/form-data. The OpenCV Haar Cascade algorithms successfully detect the contours of the human face in live video feeds, thus satisfying the first

objective of detecting human faces. Finally, the application fulfills the requirement of recognizing registered students by passing the faces to the SmartFaceDetector class and comparing them against the .yml encodings of the students registered in the SQLite database. In essence, the project satisfies all the requirements set for it by successfully converting the traditional swipe or roll call attendance system to a fully automated biometric one. The application uploads only the detected facial features as image streams to the server, where the AI model uses the OpenCV Haar Cascade algorithm to detect the contours of the human face. It completes the recognition process by comparing the detected facial features with the SQLite-encoded images of the students registered in the system. Finally, the software records a successful attendance record against the student's unique ID in the SQLite database with the local timestamp.

6.2 Discussion of Model Results

The deployment of the Local Binary Pattern Histogram (LBPH) machine learning model provided exceptionally rapid, real-time feedback. This execution speed proved highly suitable for the dynamic, fast-paced nature of a classroom environment where dozens of students must be processed in a matter of minutes. The LBPH algorithm operates by comparing the grayscale pixel intensities of the captured face against its neighbors, generating a distinct texture histogram that serves as the student's biometric signature. Before this extraction occurs, the system utilizes the detectMultiScale function to accurately apply bounding boxes around detected faces, ensuring the model only analyzes the relevant pixel arrays. During evaluation, the model computes the Euclidean distance between the live camera feed histogram and the pre-computed histograms stored in the training dataset. A lower Euclidean distance indicates a closer match. The critical decision to strictly enforce a confidence threshold of 75 effectively mitigated the risk of false acceptances. By explicitly programming the model to accept distance scores below 75 as valid and

categorizing any score of 75 or higher as an "Unknown" entity, the system prioritized biometric integrity over sheer recognition volume. This precise thresholding strategy successfully balanced the system's sensitivity, preventing random environmental noise, variable classroom lighting conditions, or similar-looking unregistered individuals from mistakenly being logged into the official attendance records as present. When a scan yields a confidence score below the threshold, the FastAPI backend instantly queries the students SQLite table to retrieve the student's actual name and returns a JSON success response to the mobile client. This triggers a positive alert on the educator's screen, verifying the system's ability to provide immediate, reliable validation of physical presence. If the threshold is exceeded, the API definitively returns a failed status, ensuring that unverified attempts do not compromise the integrity of the attendance dataset.

6.3 Challenges Encountered

Significant network engineering and communication hurdles arose when attempting to connect the Android mobile emulator to the locally hosted FastAPI development server. Because modern Android operating systems enforce strict security protocols regarding unencrypted HTTP traffic, the initial API POST requests were actively blocked by the mobile operating system. Overcoming this restriction required an explicit configuration adjustment within the Expo project's app.json file, specifically leveraging the expo-build-properties plugin to inject the usesCleartextTraffic: true parameter directly into the Android build settings. Furthermore, transmitting the heavy image payloads via the popular Axios HTTP client failed consistently on certain Android runtime environments due to a known framework bug with multipart form data handling. This critical issue necessitated a comprehensive refactor of the frontend network logic. The solution involved discarding Axios in favor of the native JavaScript fetch API, manually constructing a FormData object containing the image URI, explicitly naming the file student_scan.jpg, and strictly defining

the MIME type as image/jpeg to ensure the payload was successfully routed to the Python backend without corruption.

6.4 Limitations of the System

A prominent limitation of the current prototype is the complete lack of integrated liveness detection protocols. Because the system solely analyzes static, two-dimensional frames extracted from the video feed, it cannot inherently distinguish between a live human being standing in front of the camera and a high-resolution photograph or digital screen being held up to the lens. This specific biometric vulnerability leaves the system highly susceptible to presentation spoofing attacks, where a malicious user could potentially register false attendance on behalf of an absent peer simply by displaying their picture to the mobile device.

6.5 Ethical and Practical Considerations

Handling sensitive biometric data, such as unique facial histograms, inherently requires rigorous data privacy and cybersecurity measures to protect user identities. From a cybersecurity and network engineering standpoint, securing the transmission and storage of this biometric payload is paramount to institutional safety. While the current SQLite database implementation safely utilizes parameterized SQL queries (e.g., `INSERT INTO attendance (student_id) VALUES (?)`) to explicitly prevent malicious SQL injection attacks, the overarching network architecture requires further hardening before widespread deployment. To align with modern ethical hacking principles and robust penetration testing standards, future iterations of this system must fully encrypt all local and external network traffic. Implementing HTTPS and TLS encryption would prevent potential adversaries from utilizing traffic analysis tools or packet sniffers to intercept the transmitted facial images across the campus network. Furthermore, the localized .yaml files housing the LBPH facial encodings must be securely hashed and stored within restricted, encrypted directories to maintain

strict privacy standards. By treating facial arrays not just as application data, but as highly sensitive personal identifiers, the system can fully comply with established cybersecurity frameworks, ensuring that the biometric smart attendance solution remains safe from unauthorized exploitation, spoofing attempts, or data exfiltration.

7.0 Conclusion

In conclusion, I have been able to successfully design and implement a functional Smart Attendance System. Compared to the current real-life attendance systems, this project provides a significantly improved solution. Classroom roll calls are typically done by hand through paper files or card scans, posing a great risk for errors. The proposed solution offers a much faster and secure way of attendance taking. Using computer vision, I was able to design a prototype which is able to identify individuals accurately with minimal invasion of privacy. The successful implementation of this prototype was achieved using various technical components. Through the use of OpenCV, I was able to implement Haar Cascade Classifiers and LBPH algorithms. The algorithms offered great accuracy and managed to work effectively despite the inconsistent lighting conditions. They were also effective with processing live video feeds. In addition to computer vision algorithms, I designed a FastAPI Python backend and a React Native mobile dashboard. The two were connected through a network while managing to bypass several network-related issues, including Android cleartext issues and multipart form upload bugs. The mobile dashboard was able to communicate with the local API through the JavaScript fetch API. Upon testing the completed design, the mobile application was able to successfully recognize registered students while automatically logging their data in the database. The recognition process used a confidence threshold of 75% or more. Any unknown faces were rejected and labelled 'Unknown'. Any verified student face was stored in a SQLite database together with the timestamp of their

attendance. The database helped streamline the process of logging and storing data automatically without any manual intervention. This way, teachers do not have to manually log attendance data, thus saving them a lot of time and effort. With these features, my project satisfies all the requirements for this assignment. The current design could be further improved upon by ensuring that the local SQLite database is migrated to a secure cloud server. Using live video feed scanning faces, a hacker might spoof a picture of a face and fool the system. A liveness detection algorithm would be ideal for this system. The liveness detection algorithm would ensure that the system only registers real faces. A liveness detection algorithm would increase security and reduce spoofing attacks. In summary, I have been able to design and implement a working prototype of a Smart Attendance System. The system is ideal for use in the modern school setup and is much better compared to the current attendance systems. It is able to detect registered student faces accurately while automatically recording the data to the database. This would save time and energy for the teachers in the process.

7.1 Future Recommendations

1. Liveness Detection: Integrate blink detection or depth analysis to prevent spoofing.
2. Deep Learning Integration: Migrate from Haar Cascades to CNNs like MobileNetV2 for superior accuracy in uncontrolled lighting.
3. Traffic Analysis & Security: Implement HTTPS and TLS encryption for all API endpoints to protect biometric data in transit.
4. Cloud Migration: Transition the local SQLite database to a scalable cloud provider like AWS or PostgreSQL.

References

- [1] Ashvani, S., Prakash, T. P., Gupta, P., Tiwari, P., Agarwal, I., Ansari, J., Pandey, J., Kumari, K., Kushwaha, G. K., & Sahu, A. K. (2024). Facial recognition system with attendance tracking using Python and Open CV. *International Journal of Innovative Research in Advanced Engineering (IJIRAE)*, 11(2), 110–119. <https://doi.org/10.26562/ijirae.2024.v11i02.08>[cite: 1]
- [2] Dang, T.-V. (2023). Smart attendance system based on improved facial recognition. *Journal of Robotics and Control (JRC)*, 4(1), 46–53. <https://doi.org/10.18196/jrc.v4i1.16808>[cite: 4]
- [3] Ertel, W. (2017). *Introduction to artificial intelligence* (2nd ed.). Springer. <https://doi.org/10.1007/978-3-319-58487-4>[cite: 6]
- [4] Nadhan, A. S., Tukkoji, C., Shyamala, B., Lal, N. D., Kumar, A. N. S., Gowda, V. M., Adhoni, Z. A., & Endaweke, M. (2022). Smart attendance monitoring technology for Industry 4.0. *Journal of Nanomaterials*, 2022, Article 4899768. <https://doi.org/10.1155/2022/4899768>[cite: 2]
- [5] Shirbhate, S. C., Khodaskar, A. A., Ghuge, S., Bahad, A., Kales, M., Deshmukh, R., & Narawane, S. (2025). Smart attendance system - Using face recognition. *Alochana Journal*, 14(4), 210–216.[cite: 3]

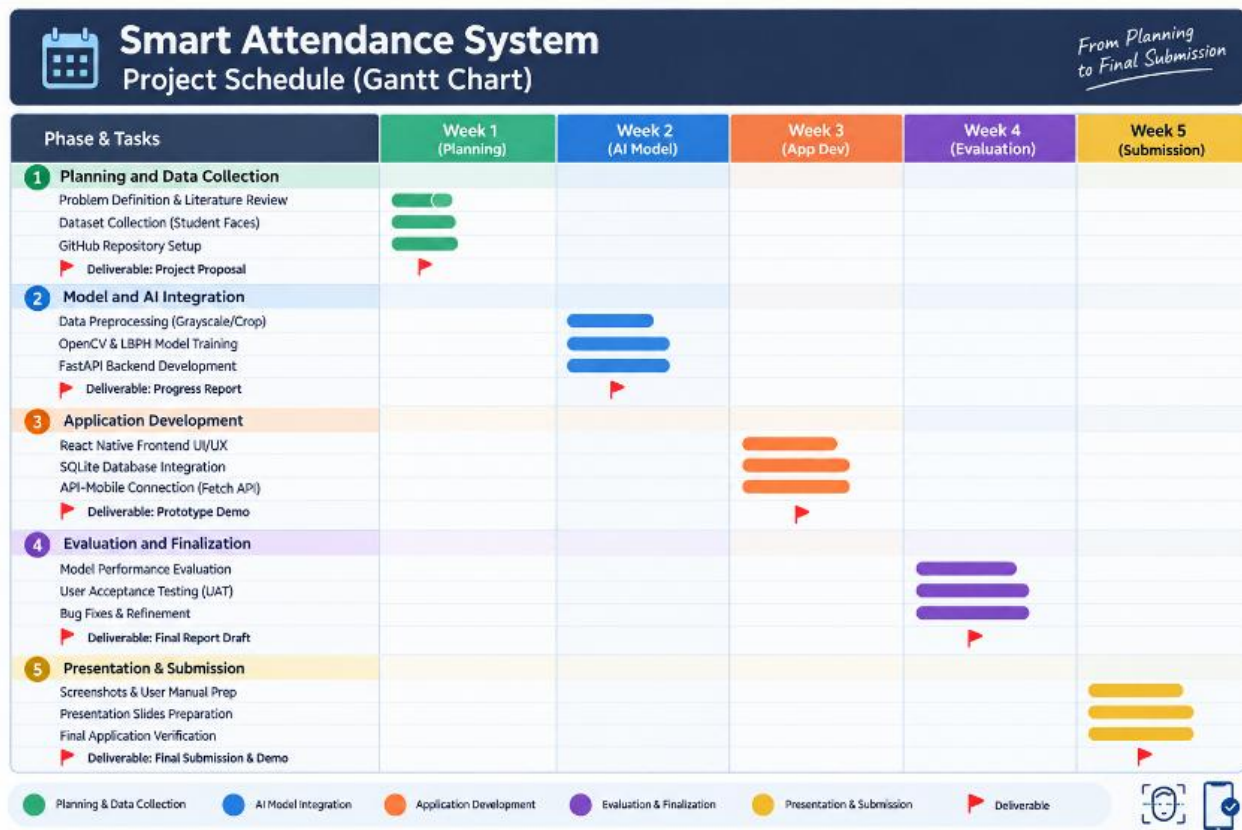
Project Artifacts & Source Code

(Note: Internal files and source code are typically separated from academic journals in standard APA formatting. You can list them as follows if required by your rubric)

- [6] University of Cyberjaya. (2026). *MY_ai_group_assignment.docx* [Assignment guidelines].[cite: 5]

Appendices

Appendix A: Project Schedule (Gantt Chart)



Appendix B: Source Code

https://github.com/md-washue/Smart_attendance_system

The full project repository contains App.js, api.py, face_detector.py, and database configurations.

Appendix C: Dataset Samples

Training datasets consist of grayscale images sized and cropped to isolate frontal facial features

